3

Universidad Nacional Autónoma de México

Facultad de Ciencias

Base de datos musical

Segundo proyecto de Modelado y Programación

Autor:
Evan Miranda Martínez

Octubre 2026

1. Resumen

Este es el contenido del resumen.

2. Introducción

El objetivo principal de este proyecto es diseñar e implementar una **base de datos** musical para archivos de audio en formato **MP3** con etiquetas **ID3v2.4**. El sistema cuenta con al menos 4 módulos bien definidos: un **minero**, una **base de datos**, un **lenguaje de consultas** y una **interfaz gráfica**.

Alcance

8-Track contempla cubrir el flujo de la especificación del **Proyecto 2**, es decir: escaneo de directorios con archivos MP3, base de datos con la biblioteca musical, edición de metadatos en la base de datos, búsquedas avanzadas a partir de un **DSL** y una intefaz gráfica que permita realizar todas las *features* de la especificación.

Adicionalmente, se contempla extender las funcionalidades básicas mediante la incorporación de reproducción de audio directamente desde la aplicación, así como el diseño de una interfaz intuitiva, visualmente atractiva y orientada a ofrecer una experiencia de usuario fluida y enriquecida.

Limitaciones

La principal limitación del proyecto es su compatibilidad restringida con formatos de audio, pues únicamente admite archivos **MP3**. En consecuencia, las colecciones musicales heterogéneas que incluyan formatos populares como **FLAC**, **ALAC**, **WAV** u **OGG** no podrán administrarse en su totalidad. Por ello, la incorporación de soporte para formatos adicionales representa una de las principales oportunidades de expansión del sistema.

Además, la aplicación opera bajo el principio de *Garbage In, Garbage Out* (GIGO), pues la calidad y precisión de la información presentada dependen directamente de los **metadatos** contenidos en los archivos **MP3**. En consecuencia, elementos como los nombres de álbumes, las portadas y la información de los artistas pueden resultar incompletos, incorrectos o incluso inexistentes si los archivos originales carecen de los datos correspondientes.

Esta dependencia podría mitigarse mediante la integración de **APIs de terceros**, como las proporcionadas por **MusicBrainz**, **Last.fm** o **Deezer**, que permitirían consultar fuentes externas para complementar, verificar y enriquecer los metadatos de la biblioteca musical, así como recuperar portadas de álbumes e imágenes de artistas. Sin embargo, estas integraciones quedan fuera del alcance actual del proyecto y se consideran posibles extensiones futuras.

El proyecto está completamente orientado a usuarios de **Linux** y no contempla una extensión multiplataforma.

3. Análisis y Diseño

3.1. Arquitectura General

La arquitectura de **8-Track** se diseñó siguiendo un enfoque modular, en el que las distintas partes del programa se distribuyen entre bibliotecas independientes. Cada biblioteca cumple un rol especifico dentro de la funcionalidad del programa:

- **Domain**: Representa las entidades básicas del programa, i.e, **canciones**, **álbumes**, **intérpretes**, entre otros.
- **Database**: Abstrae la comunicación con la base de datos por medio de interfaces y **repositorios**.
- **Compiler**: Contiene la implementación completa del compilador de **MuQL**.
- **Miner**: Realiza el escaneo de directorios para minar **metadatos** y **carátulas** de archivos **MP3**.

- **Player**: Ejecuta la reproducción de audio por medio de **QtMultimedia**.
- **Services**: Representa una capa de unificación y coordinación de las otras bibliotecas para ofrecer métodos sencillos de consumir por las vistas del programa.
- **CLI**: Es la interfaz secundaria del programa, que permite ejecutar operaciones básicas sobre la biblioteca musical mediante comandos de texto. Su propósito es ofrecer un mecanismo de interacción sencillo y directo, sin necesidad de utilizar un entorno gráfico.
- **GUI**: Es la interfaz principal del programa, diseñada para proporcionar una experiencia de usuario más intuitiva y visual. Permite interactuar con las funcionalidades del sistema mediante componentes gráficos, facilitando la navegación, consulta y administración de la biblioteca musical.

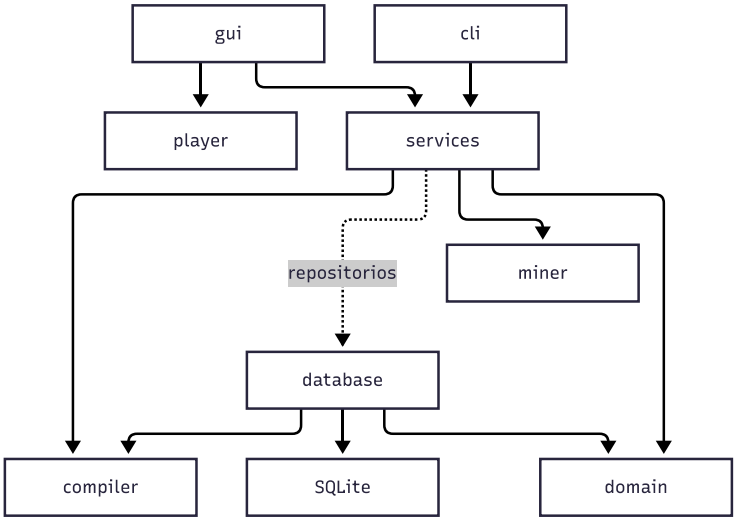


Figura 1: Diagrama de Arquitectura

La separación en bibliotecas busca reducir el acoplamiento entre componentes y delimitar sus responsabilidades. La interfaz gráfica y la interfaz de línea de comandos comparten una capa de servicios que expone las operaciones de la aplicación y coordina los subsistemas necesarios. Esto permite reutilizar la lógica de la aplicación y realizar cambios internos de manera más sencilla.

3.2. Base de Datos

Modelo Relacional

Modelos de Dominio

Repositorios

Conexión

3.3. Minero

La biblioteca utilizada para la extracción de metadatos es **TagLib**, principalmente porque ofrece una facilidad para ampliar el sistema a otros formatos de audio. A pesar de la existencia de bibliotecas más ligeras y enfocadas puramente en las etiquetas `ID3v2.4`, mi principal objetivo a futuro con **8-Track** es el soporte de archivos **Hi-Res**.

Extracción de Metadatos

La extracción de metadatos ocurre de manera recursiva buscando archivos cuya extensión de formato sea exactamente `.mp3`. Para cada archivo se extraen título, artista, álbum, género, año y número de pista. Para etiquetas faltantes, se utilizan los siguientes `fallbacks`:

- **Texto**: «Unknown»
- **Año**: 2026
- **Número de Pista**: 1

Para desacoplar el minero de la base de datos y mantener la simplicidad en las pruebas unitarias, el método principal `mine` recibe como argumento una función *callback*. De este modo, el minero permanece aislado y el comportamiento asociado a cada resultado se define externamente: por ejemplo, imprimir los metadatos, acumularlos en memoria o guardarlos directamente en la base de datos.

Por simplicidad, el reescaneo vuelve a leer los metadatos de todos los archivos `.mp3` del directorio, aunque ya estén registrados en la biblioteca, y no notifica a la base de datos sobre los archivos que fueron eliminados. Además, si no es posible extraer los metadatos de un archivo, este simplemente se ignora.

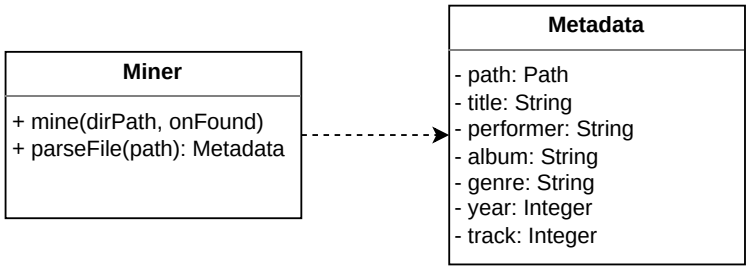
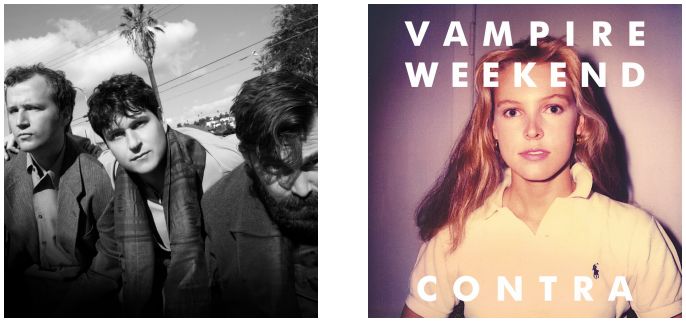


Figura 2: Diagrama UML del minero

Extracción de Carátulas

La extracción de carátulas se realiza mediante un segundo minero, `CoverMiner`. Este se encarga de extraer los *frames* `APIC` incrustados en los archivos `.mp3` de la biblioteca, identificar cuáles corresponden al **artista** y al **álbum**, y guardarlas en disco.



Artista **Álbum**

Figura 3: Carátulas de artista y álbum

Primero, el minero comprueba si la imagen ya existe en la caché. Si no se encuentra, busca un *frame* `APIC` dentro del archivo `.mp3` y selecciona la imagen de mayor jerarquía, con el siguiente orden de prioridad:

- **Álbum**: `FrontCover` → `Other`
- **Artista**: `Artist` → `LeadArtist` → `Band`

Si no existe ninguna imagen, el escaneo continúa y no se guarda nada en la caché. Para evitar búsquedas repetidas, el minero recuerda durante el escaneo los artistas y álbumes que ya intentó resolver sin éxito.

Caché de Carátulas

Las imágenes extraídas se almacenan en el directorio `covers`, dentro de la ubicación de datos de la aplicación.

Cada imágen se identifica con una clave sobre la cual se aplica una función hash **SHA-256**, cuya representación hexadecimal se utiliza como nombre del archivo.

Tipo	Clave
Álbum	`<ruta de la carpeta>|<nombre del álbum>`
Artista	`<nombre del artista>`

Cuadro 1: Construcción de la clave de cada imagen

`CoverStore` permite recuperar la ruta de una imagen recalculando su *hash*, ya sea a partir de la ruta de la carpeta y el nombre del álbum, o a partir del nombre del artista. La ruta se calcula incluso cuando el archivo no existe, por lo que verificar su existencia le corresponde a quien la consulta.

3.4. MuQL

MuQL (`Musical Query Language`) es un lenguaje de consulta para la aplicación **8-Track**, ampliamente basado en los lenguajes de búsqueda `clave-valor` de **Github** y en la sintáxis clara, legible, y cercana al lenguaje natural de **Python**.

Gramática

Una `query` de **MuQL** se compone de una expresión y una cláusula de orden. Ambos campos son opcionales y la consulta vacía es válida.

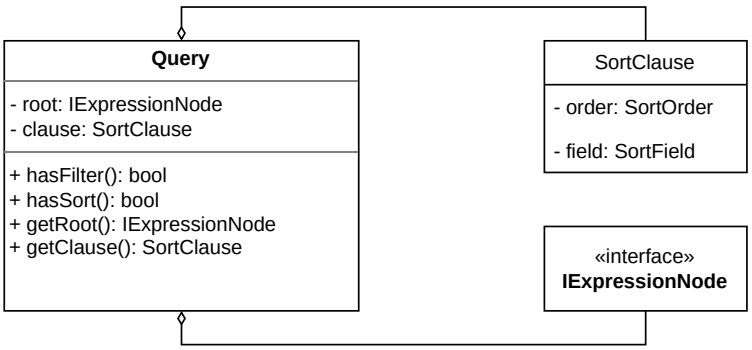


Figura 4: Diagrama UML de una Query

Coincidencias Parciales y Exactas

MuQL proporciona dos operadores para realizar búsquedas dentro de la biblioteca musical. El operador «`:`» realiza una búsqueda parcial, equivalente al `LIKE` de `SQL`: basta con que el valor aparezca en cualquier posición del campo. El operador «`:=`» realiza una búsqueda exacta, equivalente al `=` de `SQL`, que exige una coincidencia completa.

Filtros por Categoría

Los filtros por categoría restringen la búsqueda a un atributo o relación específica de la biblioteca.

- `performer`: Busca canciones por el nombre del intérprete, ya sea una persona o una banda.
- `artist`: Busca canciones por el nombre de un intérprete clasificado como persona.
- `band`: Busca canciones por el nombre de un intérprete clasificado como banda.
- `album`: Busca canciones por el nombre del álbum al que pertenecen.

- `song`: Busca canciones por su título.
- `genre`: Busca canciones por su género musical.
- `member`: Busca canciones de grupos por el nombre de uno de sus miembros.
- `path`: Busca canciones por la ruta del archivo de audio.
- `track`: Busca canciones por su número de pista dentro del álbum; admite valores individuales y rangos.

Rangos

Para los campos numéricos, como el año o el número de pista, **MuQL** ofrece el operador «`=>`», que permite filtrar por un intervalo de valores. Ambos límites son inclusivos y cualquiera de ellos puede omitirse, dejando el rango abierto por ese lado. La Tabla 2 resume las formas admitidas.

Forma	Ejemplo	Significado
Valor individual	`year: 2020`	Solo el año 2020
Rango cerrado	`year: 1990 => 1999`	Desde 1990 hasta 1999
Sin límite superior	`year: 2010 =>`	Desde 2010 en adelante
Sin límite inferior	`year: => 1980`	Hasta el año 1980

Cuadro 2: Formas de especificar valores numéricos en MuQL

Filtros Temporales

Los filtros temporales restringen la búsqueda únicamente al atributo `year`. Tanto `from` como `to` son azúcar sintáctica de rangos abiertos sobre `year`.

- `from`: Busca canciones desde un determinado año.
- `to`: Busca canciones hasta un determinado año.
- `year`: Busca canciones en un rango de años.

Búsquedas Libres

Cuando el usuario no especifica un campo, **MuQL** interpreta el término como una búsqueda libre. Una búsqueda libre se interpreta por coincidencia parcial sobre los campos: `performer`, `album`, `song` y `genre`.

Los valores que contienen caracteres especiales o acentuados, las frases completas con espacios y los valores que coinciden con palabras reservadas deben delimitarse con comillas dobles

Lógica

MuQL permite combinar expresiones mediantes los operadores booleanos `and`, `or` y `not`.

Operador	Significado	Ejemplo
`and`	Exige que se cumplan ambas condiciones.	`genre: "Rock" and year: 1990 => 1999`
`or`	Exige que se cumpla al menos una condición.	`genre: "Jazz" or genre: "Blues"`
`not`	Niega la condición siguiente.	`not album: "Live"`

Cuadro 3: Operadores lógicos de MuQL

Las expresiones consecutivas se combinan con un `and` implícito, por lo que omitir el operador no modifica el resultado de la consulta. Así, las siguientes consultas son equivalentes:

```
performer: "Vampire Weekend" and year: => 2020
performer: "Vampire Weekend" year: => 2020
```

El operador `or` permite concatenar varios valores para un mismo campo sin tener que repetirlo. Por ejemplo, las siguientes consultas son equivalentes:

```
performer: "The Beatles" or "The Zombies"
performer: "The Beatles" or performer: "The Zombies"
```

Tipo	Campos
Textual	`song`, `performer`, `album`, `genre`, `path`
Numérico	`year`, `track`

Cuadro 4: Campos disponibles para ordenar

Cláusulas de Orden

MuQL permite añadir una cláusula de ordenamiento al final de la *query*, que ordena los resultados de forma **ascendente** (`asc`) o **descendente** (`desc`) según alguno de los campos de la Tabla 4.

Se utiliza la sintáxis `sort: order by field` para especificar las claúsulas de orden. El orden por omisión es **ascendente**.

Para más ejemplos del lenguaje **MuQL**, véase `https://github.com/evanrock520-ciencias/8-Track/blob/main/docs/muql/examples.md`.

Compilador

3.5. Servicios

La capa de **servicios** actúa como una *fachada* que coordina los módulos y expone operaciones de alto nivel para la interfaz gráfica y la interfaz de línea de comandos.

Servicio de Biblioteca

`LibraryService` es la fachada de la biblioteca: coordina el minero, las carátulas, los repositorios y las transacciones, de modo que la interfaz no necesita conocer ninguno de ellos.

El escaneo comienza con `scan()`, que recorre el directorio con el minero y, por cada archivo encontrado, invoca `importTrack()`. Esta operación resuelve el intérprete y el álbum, reutilizando los existentes o creando nuevos si hace falta, construye la canción y la guarda en el repositorio. La Figura 5 muestra este flujo.

Para informar a la interfaz del avance, el servicio recibe un *callback* `ScanProgress` que reporta el archivo actual, los archivos procesados y el total.

Además del escaneo, el servicio ofrece consultas de apoyo: listado de canciones, álbumes y géneros, canciones por álbum o por intérprete, y estadísticas de la biblioteca.

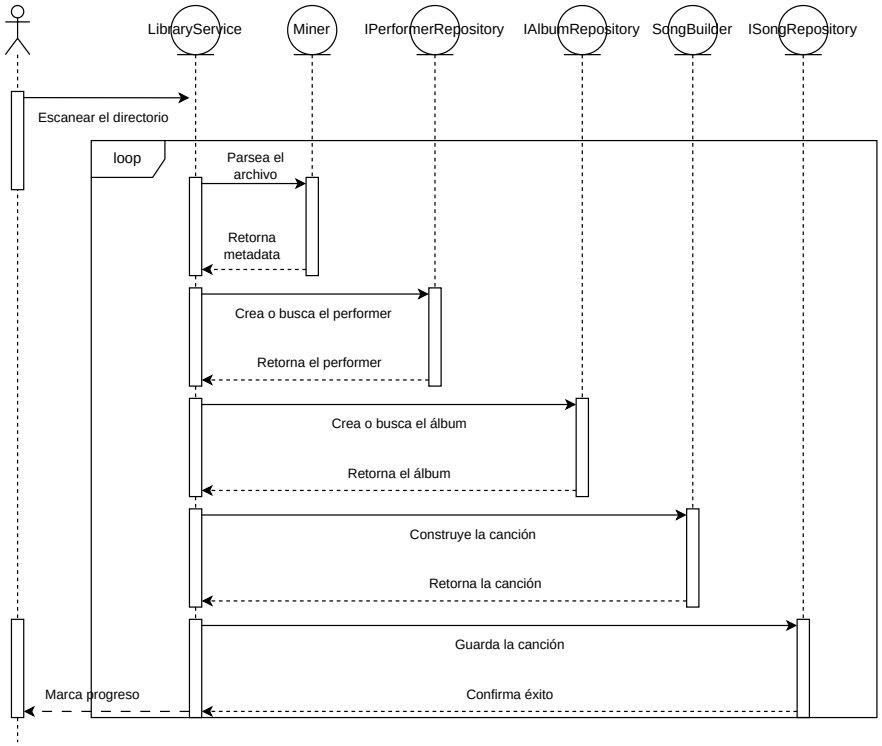


Figura 5: Diagrama de secuencia de la importación de la biblioteca

Servicio de Búsqueda

`Search Sequence` es la fachada para las búsquedas relacionadas a **MuQL**. Es una capa muy delgada que recibe una cadena cruda de código **MuQL** y retorna un **Search Result** con las canciones o un mensaje de error, con el objetivo de capturar las excepciones antes de que lleguen a la interfaz gráfica. Sigue el flujo de la Figura 6

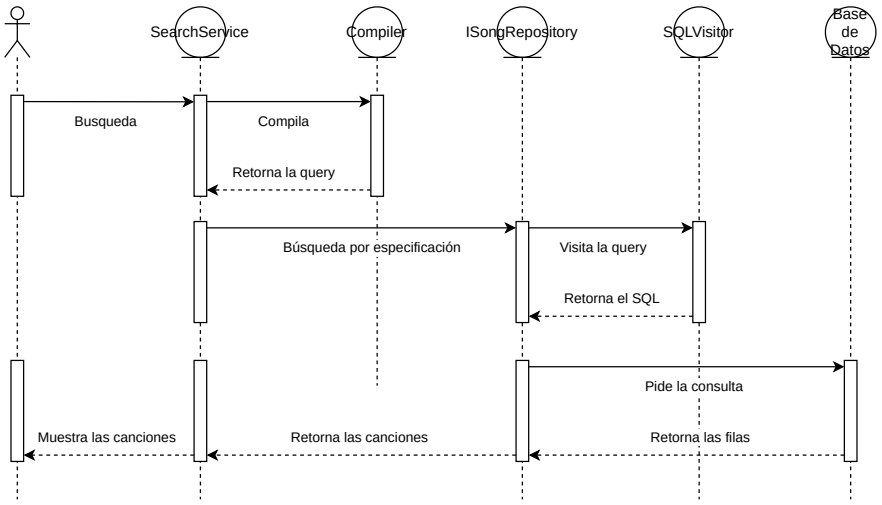


Figura 6: Diagrama de secuencia de la importación de la biblioteca

Servicio de Artistas

El servicio `ArtistService` expone las operaciones de consulta y edición de intérpretes. Permite clasificarlos como personas, grupos o desconocidos, actualizar sus datos y administrar las pertenencias entre personas y grupos.

Estas operaciones se delegan en `IPerformerRepository`, cuya implementación se encarga de la persistencia.

4. Implementación

4.1. Requisitos y Tecnologías

El desarrollo de **8-Track** se llevó a cabo en el lenguaje de programación **C++20** y contó con el uso de múltiples dependencias del framework **Qt**.

Tecnología	Versión	Uso
C++	20	Lenguaje de implementación
QtQuick	Qt 6.8+	Interfaz gráfica
QtSQL	Qt 6.8+	Acceso a la base de datos
QtMultimedia	Qt 6.8+	Reproducción de audio
SQLite	vía QSQLITE	Motor de base de datos

Cuadro 5: Tecnologías del núcleo de la aplicación

Todas las dependencias `third-party` se descargan automáticamente antes de la compilación por medio de `FetchContent`; por lo tanto, es necesario contar con conexión a internet.

4.2. Organización del Proyecto

El código de la aplicación se concentra en `8-track/`. Las cabeceras de C++ se agrupan por responsabilidad en `include/`, mientras que `src/` contiene sus implementaciones y los puntos de entrada de la CLI y la interfaz gráfica.

Herramienta	Versión	Uso
CMake	3.25+	Configuración y generación del proyecto
Ninja	—	Generador del preset de release
ccache	—	Caché de compilación
mold	—	Enlazador del preset de release

Cuadro 6: Herramientas de construcción

Biblioteca	Versión	Uso
TagLib	2.3.2	Lectura de las etiquetas ID3v2.4 y carátulas
PicoSHA2	1.0.1	Hash para el caché de las carátulas
argparse	3.2	Argumentos para la CLI
tabulate	2.0	Presentación de datos tabulados para la CLI
indicators	2.3	Barra de progreso para la CLI

Cuadro 7: Bibliotecas de terceros utilizadas por la aplicación

```
./
├── 8-track/ ..... Código y recursos de la aplicación
│   ├── include/ ..... Cabeceras de C++ por módulo
│   │   ├── app/ ..... Ensamblado de repositorios y servicios
│   │   ├── cli/ ..... Comandos y presentación de tablas
│   │   ├── compiler/ ..... Análisis de MuQL y traducción a SQL
│   │   ├── database/ ..... Conexión y repositorios SQLite
│   │   ├── domain/ ..... Entidades y tipos del dominio musical
│   │   │   └── repositories/ ..... Interfaces de persistencia y transacciones
│   │   ├── gui/ ..... Integración de C++ con la interfaz QML
│   │   │   └── models/ ..... Modelos de listas y controladores Qt
│   │   ├── miner/ ..... Extracción de metadatos y caché de carátulas
│   │   ├── player/ ..... Cola de reproducción y motor de audio
│   │   ├── services/ ..... Biblioteca, búsquedas y gestión de intérpretes
│   │   └── utils/ ..... Funciones auxiliares de texto y del compilador
│   ├── src/ ..... Implementaciones de C++ y puntos de entrada
│   ├── resources/ ..... Recursos del núcleo de la aplicación
│   │   └── sql/ ..... Esquema de la base de datos SQLite
│   └── qml/ ..... Interfaz gráfica y ventana principal
│       ├── components/ ..... Controles y elementos visuales reutilizables
│       ├── dialogs/ ..... Selector de directorios para el escaneo
│       ├── pages/ ..... Vistas de canciones, álbumes, artistas y búsqueda
│       ├── resources/ ..... Iconos, imágenes y fuentes tipográficas
│       ├── theme/ ..... Colores, tipografías y medidas de la interfaz
│       └── utils/ ..... Formato del tiempo de reproducción
└── tests/ ..... Pruebas por módulo con GoogleTest
    └── data/ ..... Archivos MP3 e imágenes para las pruebas
```

Herramienta	Versión	Uso
GoogleTest	1.15.2	Pruebas unitarias
prek	0.5	Gestión de hooks de Git
clang-format	23.1.1	Formato del código C++
qmlformat	1.2.0	Formato del código QML
Doxygen	1.18	Generación de documentación
GitHub Actions	—	Integración continua (CI)

Cuadro 8: Herramientas de pruebas, calidad y documentación

4.3. Instrucciones de Compilación

Se recomienda utilizar el preset `release`, que centraliza la configuración empleada durante el desarrollo y en integración continua. Este utiliza Ninja para la construcción, ccache para reutilizar resultados de compilaciones anteriores y mold para acelerar el enlazado.

Para utilizarlo, es necesario contar con estas herramientas instaladas. En **Fedora**, pueden instalarse con el siguiente comando:

```
sudo dnf install ninja-build mold ccache
```

Primero, se debe clonar el repositorio y acceder al directorio del proyecto:

```
git clone https://github.com/evanrock520-ciencias/8-Track.git
cd 8-Track
```

A continuación, se puede elegir una de las siguientes alternativas. Ambas compilan el proyecto en modo `Release`.

Compilación con preset

El preset almacena los archivos generados en `build/release`:

```
cmake -S . --preset release
cmake --build --preset release
```

Compilación sin preset

Como alternativa, se puede configurar la construcción directamente, sin exigir Ninja, ccache ni mold. En este caso, los archivos generados se almacenan en `build`:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

4.4. Instrucciones de Ejecución

Interfaz Gráfica

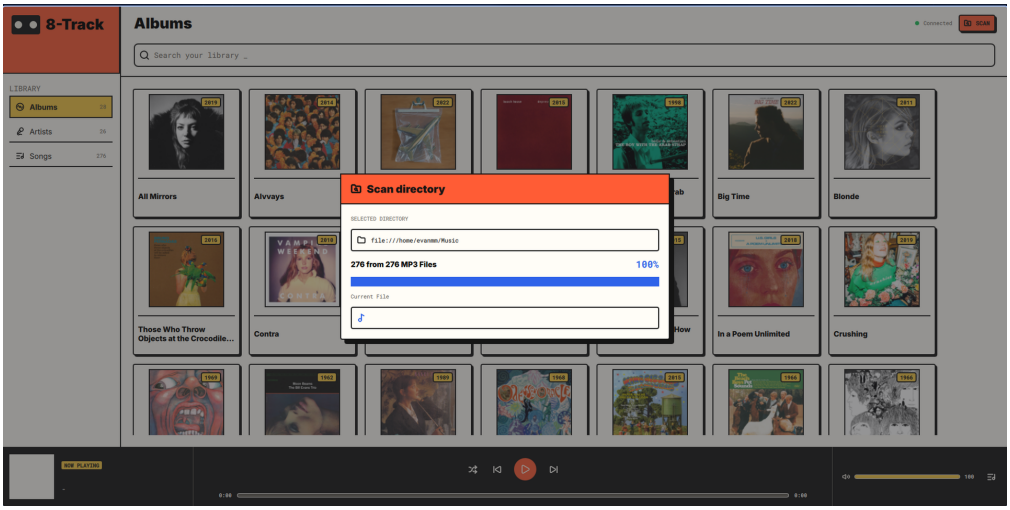
En el primer arranque, la aplicación crea automáticamente la base de datos SQLite e inicializa su esquema. Los datos se almacenan en el directorio determinado por `QStandardPaths::AppDataLocation`, que en Linux normalmente corresponde a `$HOME/.local/share/8-track/`

Abrir interfaz gráfica

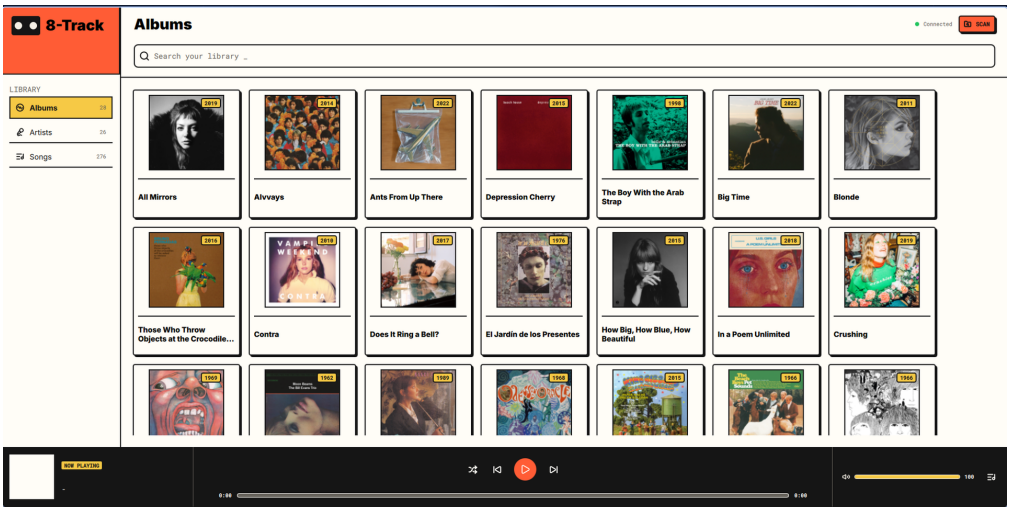
```
build/release/8-track/qml/track8
```

La biblioteca inicialmente está vacía. Para agregar canciones se debe seleccionar un directorio mediante el botón `Scan` de la interfaz gráfica o mediante el comando `--scan` de la CLI.

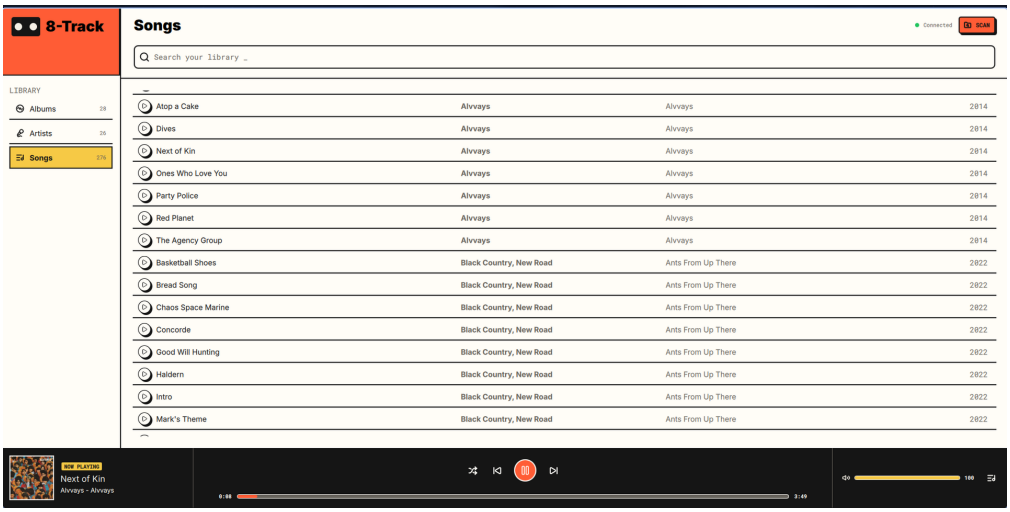
Al seleccionar el directorio, comienza el escaneo de los archivos MP3 y se muestra una ventana con el progreso y el archivo actual.



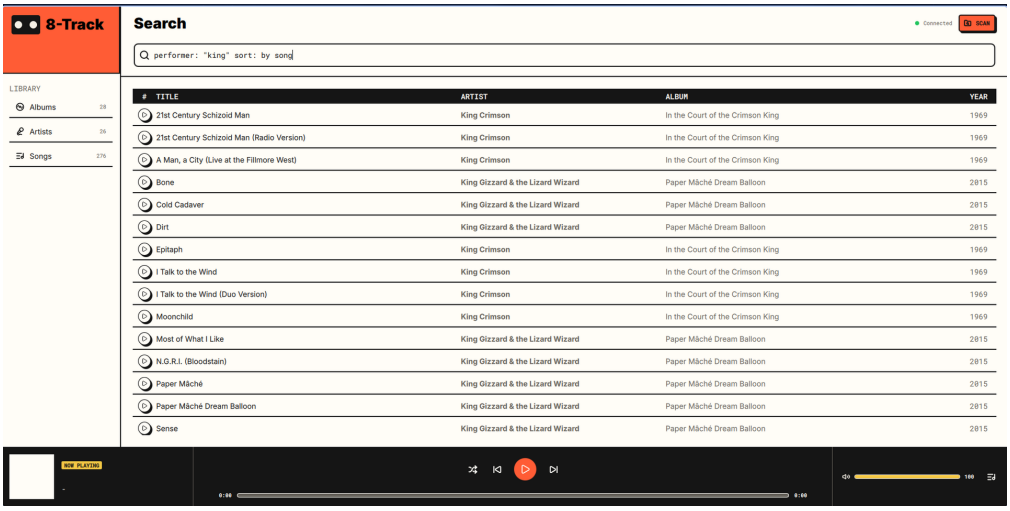
Al finalizar, la biblioteca se actualiza con las canciones, álbumes e intérpretes encontrados.



Para iniciar la reproducción, se utiliza el botón de reproducción de una canción o de un álbum. La barra inferior permite pausar o reanudar el audio, cambiar de canción, activar el modo aleatorio y ajustar el volumen y la posición de reproducción.



Para buscar canciones, se escribe una consulta de MuQL en la barra superior y se presiona `Enter`;



CLI

Escanear un directorio

```
build/release/track8-cli --scan $HOME/Music
```

Ver la biblioteca

```
build/release/track8-cli --list
```

```
+------------+---------------+-------------------+--------------+
|   Title    |   Performer   |       Album       | Release Year |
+------------+---------------+-------------------+--------------+
| Big Time   | Angel Olsen   |     Big Time      |         2022 |
+------------+---------------+-------------------+--------------+
| Everyday   | Weyes Blood   | Titanic Rising    |         2019 |
+------------+---------------+-------------------+--------------+
| Unknown    | Unknown       |     Unknown       |         2026 |
+------------+---------------+-------------------+--------------+
```

Realizar una consulta

```
build/release/track8-cli --query 'big'
```

```
+------------+---------------+------------+--------------+
|   Title    |   Performer   |   Album    | Release Year |
+------------+---------------+------------+--------------+
| Big Time   | Angel Olsen   | Big Time   |         2022 |
+------------+---------------+------------+--------------+
```

4.5. Pruebas

La mayoría del proceso de desarrollo fue guiado por pruebas unitarias desarrolladas con la biblioteca de **GoogleTest**. Las pruebas están divididas por módulos, cada uno correspondiente a una biblioteca de **8-Track**.

Suite	Cobertura
database	Implementación de los repositorios para SQLite
domain	Validación de fechas
compiler	Pruebas para lexer, parser, generación del AST y transpilación a SQL
miner	Mineria de metadatos con archivos de prueba heterogéneos
player	Lógica de la cola de reproducción.
services	Verificación de datos y acciones de usuario

Cuadro 9: Suites de pruebas unitarias

Ejecutar Pruebas

Para ejecutar todas las suites de pruebas registradas en CTest, se utiliza el siguiente comando:

```
ctest --test-dir build/release/ --output-on-failure
```

También es posible ejecutar una suite de pruebas de manera individual, invocando directamente su ejecutable. Por ejemplo, para ejecutar la suite correspondiente a la base de datos:

```
./build/release/tests/database_tests
```

4.6. CI/CD

Verificación

El `workflow` principal de `CI` sirve principalmente como una verificación de reproducibilidad del sistema. Se encarga de configurar el entorno, descargar dependencias, verificar el formato de código, compilar **8-Track** y correr las suites de pruebas unitarias.

Calidad

La calidad, legibilidad y consistencia del código es bastante importante para mí. Por ello, en lugar de depender de una disciplina manual para mantener un formato consistente, el proceso se automatiza mediante *hooks* de `prek`: `clang-format` para **C++** y `qmlformat` para **QML**. Estos se ejecutan antes de cada *commit* y se vuelven a verificar en el `workflow` principal de `CI`, de modo que ningún cambio con formato inconsistente llega a `main`.

Documentación

Durante cada `push` a la rama `main` se ejecuta un despliegue automático de documentación relacionada al código del proyecto en `https://evanrock520-ciencias.github.io/8-Track/`.

5. Conclusiones

Este es el contenido de las conclusiones.

Referencias

[1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021. URL: `https://craftinginterpreters.com/` (visitado 26-10-2026).